

GSLC Algorithm: Design & Analysis Session 5

Name: Wilbert Chandra

NIM: 2602113666

Class: LB09

1. Jelaskan pengertian ADT? Sebutkan kegunaan ADT dalam Algorithm Design and Analysis?

ADT atau *Abstract Data Types*, adalah suatu model matematis dalam struktur data yang memberikan informasi akan tipe data yang disimpan serta operasi yang dapat dilakukan kepada data tersebut. ADT berguna pada *Algorithm Design and Analysis* untuk mensimplifikasi proses pemecahan masalah (*Problem Solving*). Dengan ADT user tidak perlu mengetahui bagaimana suatu data di implementasikan, melainkan hanya kegunaan dari data tersebut.

2. Jelaskan perbedaan Stack, Queue, Tree, dan Binary Tree!

	Kenapa Digunakan?	Keuntungan	Kekurangan
Stack	Stack adalah struktur data linear yang menggunakan prinsip LIFO (<i>Last in First Out</i>), hal ini membuatnya baik untuk digunakan dalam program <i>recursive</i> , <i>undo-redo</i> , <i>backtracking</i> , dll.	Simplisitas dan efisiensi dalam <i>memory</i> dan <i>time complexity</i>	Ukuran yang terbatas, rentan <i>overflow</i> dan <i>underflow</i> , dan restriksi akses pada data lama.
Queue	Queue adalah struktur data linear yang menggunakan prinsip FIFO (<i>First in First Out</i>), contoh penggunaannya adalah <i>task handling</i> , <i>data processing system</i> , <i>network protocols</i> , dll.	Memberikan simplisitas and efisiensi dalam operasi <i>insertion</i> and <i>deletion</i> dengan data di atas.	Operasi yang memakan waktu lama pada data di tengah queue, searching yang tak efisien, ukuran yang terbatas.
Tree	Tree adalah stuktur data non linear yang merepresentasikan <i>relationship</i> hirarkis yang terdiri atas <i>node</i> yang saling terhubung. Contoh penggunaan tree adalah struktur <i>file directory</i> .	Memberikan efisiensi dalam operasi searching pada data.	Memakan memori yang cukup signifikan, tree yang bisa menjadi imbalanced, rentan menjadi sangat kompleks.
Binary Tree	Binary Tree adalah tree yang dimana setiap <i>node</i> hanya mempunyai maksimal 2 <i>child</i> . Contoh penggunaannya adalah dalam searching atau sorting , seperti <i>Binary Search tree</i> .	Memberikan efisiensi dalam operasi searching dan sorting pada data.	Memakan memori yang cukup signifikan, tree yang bisa menjadi imbalanced, evaluasi yang sequential.

- Stack and Queue

Saat menyelesaikan soal matematika rumit, kita bisa menggunakan stack untuk menghitung ekspresi postfix (notasi yang ditulis dengan operator di belakang operand). Ekspresi ini dihitung dengan cara "memasukkan" operand ke stack dan kemudian "mengeluarkan" dua operand untuk dioperasikan dengan operator yang sesuai. Hasil operasi kemudian "dimasukkan" kembali ke stack.

Sedangkan untuk ekspresi infix (notasi yang ditulis dengan operator di antara operand), kita bisa menggunakan queue. Ekspresi ini dihitung dengan cara "memasukkan" operand dan operator ke queue secara berurutan. Kemudian, operand dan operator dikeluarkan dari queue untuk dioperasikan sesuai aturan prioritas operator.

- Stack and tree

Saat menjelajahi tree, kita bisa menggunakan stack untuk menyimpan node yang akan dikunjungi selanjutnya. Hal ini membantu kita untuk melakukan traversal dengan cara yang terstruktur, seperti postorder, preorder, atau inorder.

- Tree and Queue

Saat ingin melihat semua node di tree secara berurutan dari atas ke bawah, kita bisa menggunakan queue. Kita "memasukkan" node ke queue, kemudian "mengeluarkan" dan memprosesnya satu per satu. Hal ini memungkinkan kita untuk melihat semua node pada setiap level tree dengan benar.

3. Jelaskan pengertian dan penggunaan circular ADT!

Circular ADT (Abstract Data Type) adalah struktur data yang menggabungkan konsep linked list dan circular linked list. ADT ini memiliki beberapa karakteristik:

- Struktur data linear: Data terhubung secara berurutan, seperti rantai.
- Elemen terakhir menunjuk ke elemen pertama: Membentuk lingkaran.
- Operasi insertion dan deletion dapat dilakukan di mana saja: Tidak terbatas pada awal atau akhir.
- Akses data: Membutuhkan traversal dari elemen awal.

Penggunaan Circular ADT:

- Implementasi queue: Circular queue memungkinkan enqueue dan dequeue di mana saja dalam struktur data.
- Implementasi Josephus problem: Simulasi di mana orang dieliminasi secara berurutan, dan orang yang tersisa adalah pemenangnya.
- Memory management: Membantu dalam mengalokasikan dan dealokasikan memori secara dinamis.
- Graph traversal: Digunakan untuk traversal graph yang kompleks.

Keuntungan Circular ADT:

- Efisien: Insertion dan deletion dapat dilakukan di mana saja tanpa perlu shifting data.
- Fleksibilitas: Cocok untuk situasi di mana data perlu diakses dan dimodifikasi secara dinamis.
- Memori: Penggunaan memori lebih efisien dibandingkan array.

Kekurangan Circular ADT:

- Kompleksitas: Implementasi dan operasi bisa lebih kompleks dibandingkan linked list tradisional.
- Akses data: Membutuhkan traversal dari elemen awal untuk mencapai elemen tertentu.

4. Perbedaan Array dan Linked-List

Array:

- Struktur data statis yang menyimpan elemen data dengan tipe data yang sama.
- Elemen disimpan dalam lokasi memori yang berdekatan.
- Akses elemen cepat dan efisien dengan menggunakan indeks.
- Ukuran array harus ditentukan terlebih dahulu dan tidak dapat diubah.

- Kelebihan: akses data cepat, efisien untuk operasi pengindeksan.
- Kekurangan: ukuran statis, tidak efisien untuk penyisipan dan penghapusan data.

Linked-List:

- Struktur data dinamis yang menyimpan elemen data dengan tipe data yang sama.
- Elemen disimpan dalam node yang terhubung satu sama lain dengan pointer.
- Akses elemen membutuhkan traversal melalui node.
- Ukuran linked-list dapat diubah secara dinamis.
- Kelebihan: ukuran dinamis, efisien untuk penyisipan dan penghapusan data.
- Kekurangan: akses data lambat, tidak efisien untuk operasi pengindeksan.

5. Tree Transversal

Pengertian: Tree transversal adalah proses mengunjungi semua node dalam tree, dan memproses data yang disimpan dalam node tersebut.

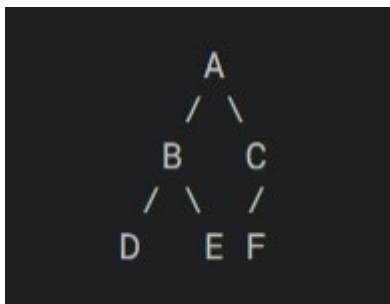
Cara Kerja:

Ada tiga metode utama untuk melakukan tree transversal:

- Preorder: Kunjungi root node terlebih dahulu, kemudian kunjungi semua node di subtree kiri, dan terakhir kunjungi semua node di subtree kanan.
- Inorder: Kunjungi semua node di subtree kiri terlebih dahulu, kemudian kunjungi root node, dan terakhir kunjungi semua node di subtree kanan.
- Postorder: Kunjungi semua node di subtree kiri terlebih dahulu, kemudian kunjungi semua node di subtree kanan, dan terakhir kunjungi root node.

Simulasi:

Misalkan kita memiliki tree berikut:



Preorder: A, B, D, E, C, F

Inorder: D, B, E, A, C, F

Postorder: D, E, B, F, C, A

6. Euler Tour Transversal

Pengertian: Euler tour transversal adalah metode tree transversal yang mengunjungi setiap edge dalam tree tepat sekali, dan memulai dan berakhir pada node yang sama.

Cara Kerja:

- Mulai dari node root.
- Jika node saat ini memiliki child yang belum dikunjungi, pilih salah satu child tersebut dan ikuti edge yang menghubungkannya.
- Jika node saat ini tidak memiliki child yang belum dikunjungi, kembali ke parent node.
- Ulangi langkah 2 dan 3 sampai semua node telah dikunjungi.

Simulasi: Misalkan kita memiliki tree seperti ini :



Euler tour transversal: A, B, D, E, B, C, F, A

7. Operasi Push dan Pop pada Stack

Push:

Push(data):

```
If top == max - 1:  
    print("Stack penuh")  
    return  
top += 1  
arr[top] = data
```

Pop:

Pop(data):

```
If top == -1:  
    print("Stack kosong")  
    return None
```

```
data = arr[top]
```

```
top -= 1
```

```
return data
```

Kompleksitas:

- Push: $O(1)$
- Pop: $O(1)$

8. Create algorithms for operation of push and pop in circular queue, and explain its complexity.

Push:

Enqueue(data):

```
if(rear + 1) % max == front:
    print("Queue penuh")
    return
elif front == -1:
    front = 0
Rear = (rear + 1) % max
arr[rear] = data
```

Complexity : memiliki proses waktu yang konstan, dan kompleksitas ruang yang konstan karna menggunakan jumlah ruang sementara

Pop:

dequeue(data):

```
if front == -1:
    print("Queue penuh")
    return
if front == rear:
    data = arr[front]
    front = rear = -1
    return data
data = arr[front]
front = (front + 1) % max
return data
```

complexity : memiliki proses waktu yang konstan, dan kompleksitas ruang yang konstan karna menggunakan jumlah ruang sementara